

AWS VPC — Complete Case Study Documentation

Module: AWS Networking | **Level:** Beginner to Production

Goal: Understand VPC from scratch, then build a full production-grade three-tier architecture on AWS.

Table of Contents

1. What is a VPC?
 2. Public Subnet
 3. Private Subnet
 4. Internet Gateway (IGW)
 5. NAT Gateway
 6. Bastion Host
 7. Route Tables
 8. Security Groups
 9. Task — Production Three-Tier Architecture on AWS
-

1. What is a VPC?

Definition

A **Virtual Private Cloud (VPC)** is your own logically isolated section of the AWS Cloud. Think of it as your **private data center inside AWS** — you control everything: IP address ranges, subnets, routing, and security.

When you sign up for AWS, you get a default VPC in every region. But for production workloads, you always create a **custom VPC** so you have full control.

How AWS handles it

- AWS assigns your VPC a **CIDR block** (Classless Inter-Domain Routing) — a range of private IP addresses.
- Everything inside the VPC stays **private by default** — no internet access unless you explicitly configure it.
- A VPC is **region-scoped** — it spans all Availability Zones within that region.
- You can create multiple VPCs per region (default limit: 5, can be increased).

Why it matters

Without VPC	With VPC
Shared network with all AWS customers	Fully isolated private network
No control over IP ranges	You define your own IP scheme
No fine-grained routing control	Custom route tables per subnet
Security is harder to enforce	Layered security with SGs and NACLs

CIDR Block — Understanding IP Ranges

```
VPC CIDR: 10.0.0.0/16
```

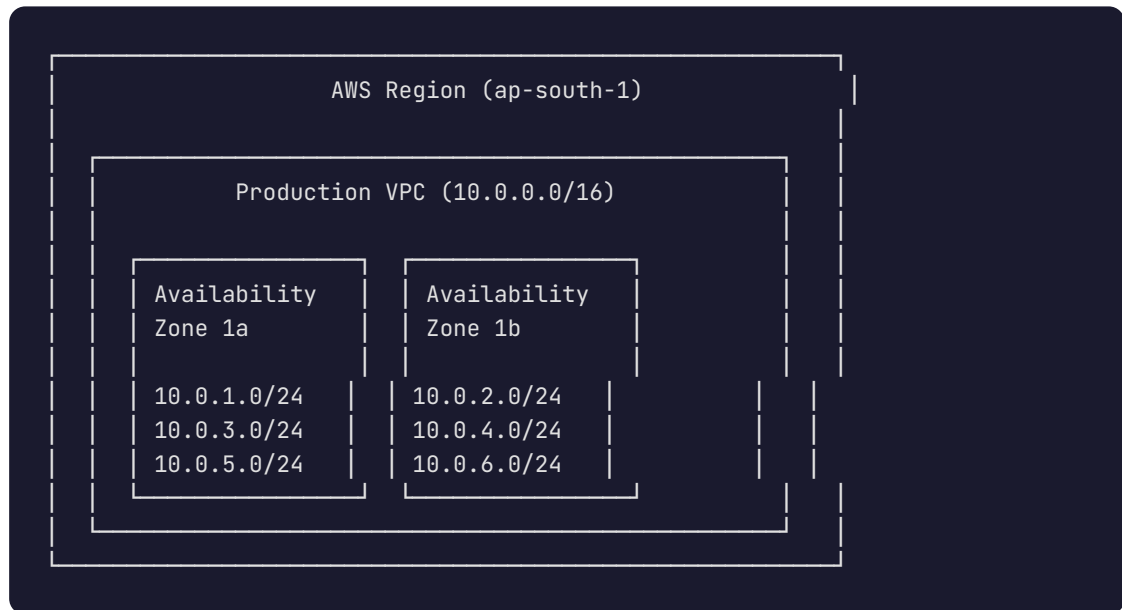
```
/16 means:
```

- First 16 bits are fixed (10.0)
- Last 16 bits are flexible
- Total IPs: 65,536

```
You then divide this into smaller subnets:
```

```
10.0.1.0/24 → 256 IPs (subnet 1)  
10.0.2.0/24 → 256 IPs (subnet 2)  
10.0.3.0/24 → 256 IPs (subnet 3)  
... and so on
```

ASCII Diagram — VPC Overview



2. Public Subnet

Definition

A **Public Subnet** is a subnet inside your VPC that has a direct route to the **Internet Gateway**. Resources placed here can have public IP addresses and communicate directly with the internet.

Characteristics

- Has a route: `0.0.0.0/0 → Internet Gateway`
- Instances can have **public IPv4 addresses** (auto-assign enabled)
- Directly reachable from the internet
- Used for resources that **need** to be internet-facing

What lives in a Public Subnet?

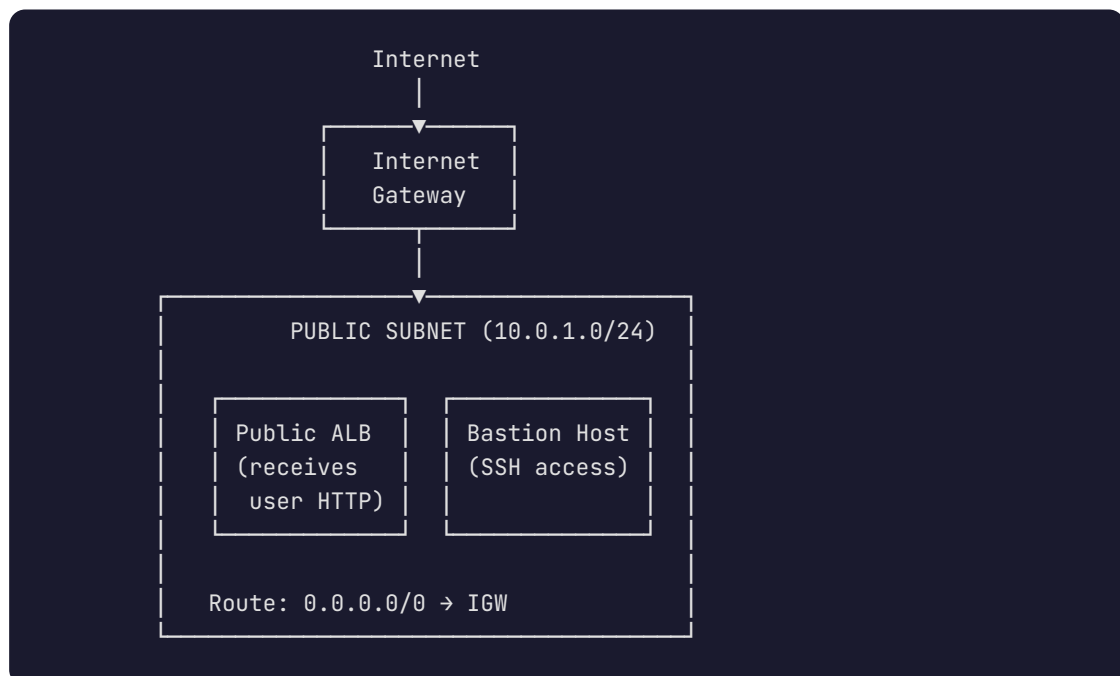
Resource	Why Public?
Application Load Balancer (ALB)	Receives traffic from users on the internet
Bastion Host	Needs to be SSH-able from your laptop
NAT Gateway	Must reach the internet to forward private traffic
Web Tier (NGINX)	Serves frontend to users

AWS Networking Detail

AWS reserves **5 IP addresses** in every subnet (first 4 and last 1): - **.0** — Network address - **.1** — VPC router - **.2** — AWS DNS - **.3** — Reserved for future use - **.255** — Broadcast address

So a **/24** subnet gives you **251 usable IPs**, not 256.

ASCII Diagram — Public Subnet



3. Private Subnet

Definition

A **Private Subnet** is a subnet with **no direct route to the internet**. Instances here have only private IP addresses and cannot be reached from the internet directly. They can only send outbound internet traffic through a **NAT Gateway**.

Why use Private Subnets?

Security is the core reason. You never want your database or application servers directly exposed to the internet. If an attacker can't reach your server, they can't attack it.

Types of Private Subnets in a Three-Tier Architecture

Subnet Type	Purpose	Example CIDR
App Private Subnet	Node.js / backend application servers	10.0.3.0/24, 10.0.4.0/24
Database Private Subnet	MySQL / RDS databases	10.0.5.0/24, 10.0.6.0/24

AWS Networking Detail

- Private subnets use the **main route table** or a custom route table with no IGW route
- Outbound internet (for updates, package installs) goes through **NAT Gateway**
- Inbound access only allowed from within the VPC (e.g., internal ALB, Bastion Host)

ASCII Diagram — Private Subnet



4. Internet Gateway (IGW)

Definition

An **Internet Gateway (IGW)** is a horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet. It serves two purposes:

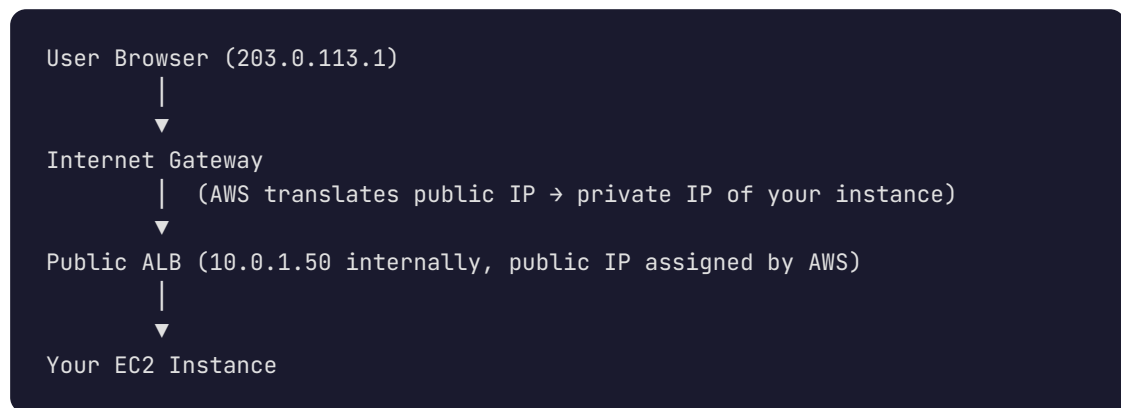
1. Provides a **target in route tables** for internet-routable traffic
2. Performs **Network Address Translation (NAT)** for instances with public IPv4 addresses

Key Facts

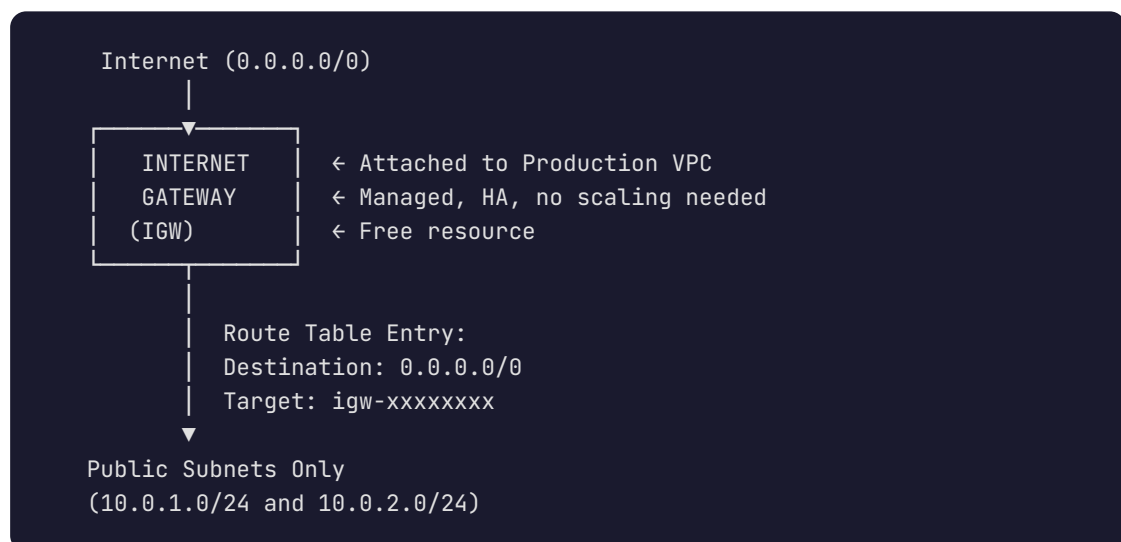
- **One IGW per VPC** — you cannot attach multiple IGWs to one VPC
- **Fully managed by AWS** — no bandwidth limits, no availability concerns, no scaling needed

- **Free** — no cost to create or attach an IGW (you pay for data transfer)
- Must be **attached** to the VPC to work (created separately, then attached)

How Traffic Flows Through IGW



ASCII Diagram — IGW Role



IGW vs NAT Gateway

Feature	Internet Gateway	NAT Gateway
Direction	Inbound + Outbound	Outbound only
Used for	Public subnets	Private subnets
Cost	Free	Paid (hourly + data)
Public IP needed?	Yes (on instance)	No (NAT has its own EIP)

5. NAT Gateway

Definition

A **NAT Gateway (Network Address Translation Gateway)** allows instances in **private subnets** to initiate outbound connections to the internet while preventing the internet from initiating connections back to those instances.

Why NAT Gateway?

Your private EC2 instances (app servers, database) still need internet access for: - `apt update` — OS security patches - `npm install` — application dependencies - `git clone` — pulling code from GitHub - Downloading SSL certificates

But you don't want the internet to reach them directly. NAT Gateway solves this.

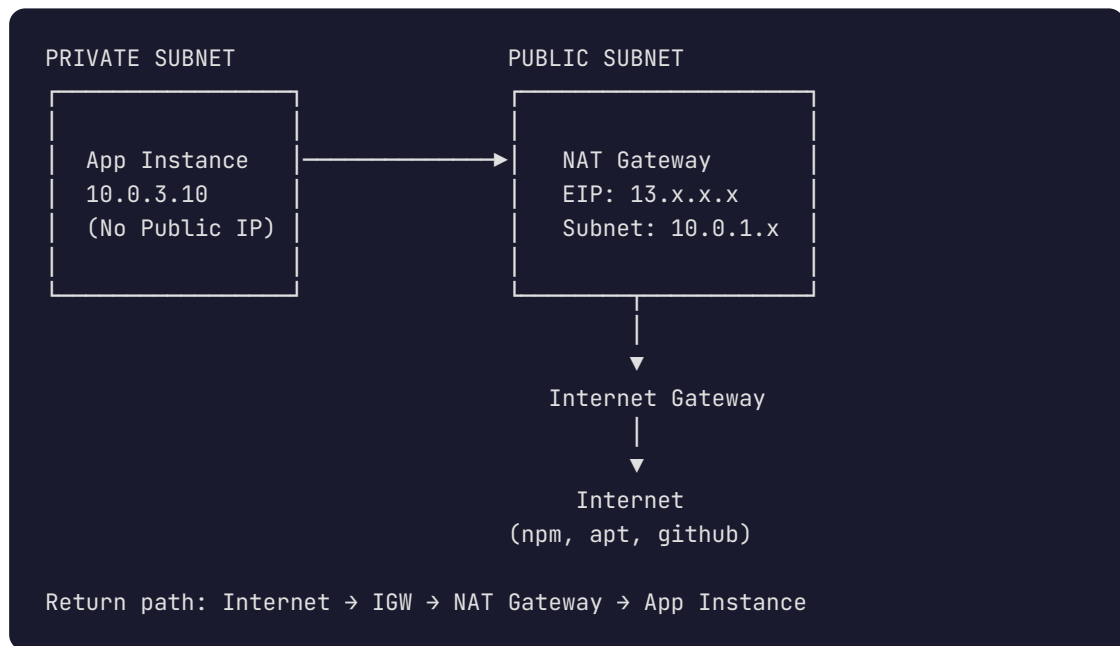
How NAT Gateway Works

1. Private instance sends traffic to NAT Gateway
2. NAT Gateway **replaces the source IP** with its own Elastic IP (public IP)
3. Internet sees only the NAT Gateway's IP — private instance stays hidden
4. Response comes back to NAT Gateway, which forwards it to the private instance

Key Facts

- **Lives in a public subnet** (needs internet access itself)
- Requires an **Elastic IP** (static public IP assigned to it)
- **Managed by AWS** — highly available within an AZ
- For multi-AZ HA: deploy one NAT Gateway **per AZ**
- **Costs money** — hourly charge + per GB data processed

ASCII Diagram — NAT Gateway Flow



NAT Gateway vs NAT Instance

Feature	NAT Gateway	NAT Instance
Management	Fully managed by AWS	You manage the EC2
Availability	Highly available in AZ	Single point of failure
Bandwidth	Up to 45 Gbps	Limited by instance type
Cost	Higher	Cheaper (EC2 cost)
Recommendation	Production	Learning/dev only

6. Bastion Host

Definition

A **Bastion Host** (also called a Jump Server or Jump Box) is a special-purpose EC2 instance placed in a **public subnet** that acts as a secure gateway for SSH access to instances in **private subnets**.

The Security Problem It Solves

Your app servers and database are in private subnets — they have no public IP and cannot be SSH'd directly from the internet. But you (the developer/admin) still need to: - Debug application issues - Run database migrations - Install software manually

The Bastion Host gives you a **single, controlled entry point** into your private network.

Security Best Practices for Bastion Host

- Allow SSH (port 22) **only from your specific IP** (not `0.0.0.0/0` in production)
- Use **key pairs** — never passwords
- Enable **CloudTrail** to log all Bastion activity
- Consider **AWS Systems Manager Session Manager** as a zero-bastion alternative
- Shut it down when not in use (to save cost and reduce attack surface)

How SSH Jump Works

```
# Step 1: SSH into Bastion Host (it's in public subnet, has public IP)
ssh -i mykey.pem ubuntu@<bastion-public-ip>

# Step 2: From Bastion, SSH into App Server (private IP only)
ssh -i mykey.pem ubuntu@10.0.3.10

# Or use SSH ProxyJump in one command:
ssh -i mykey.pem -J ubuntu@<bastion-ip> ubuntu@10.0.3.10
```

ASCII Diagram — Bastion Host Access



7. Route Tables

Definition

A **Route Table** is a set of rules (called routes) that determines where network traffic is directed within your VPC. Every subnet must be associated with a route table, which controls the routing for that subnet.

How Route Tables Work

- Every VPC has a **Main Route Table** (created automatically)
- You can create **custom route tables** and associate them with specific subnets
- Each route has a **Destination** (CIDR range) and a **Target** (where to send it)
- AWS uses the **most specific route** (longest prefix match) first

Route Table Types in Production

1. Public Route Table

Destination	Target	
10.0.0.0/16	local	← All VPC-internal traffic stays local
0.0.0.0/0	igw-xxxxxxx	← All internet traffic → Internet Gateway

Associations: web-public-subnet-1a, web-public-subnet-1b

2. Private App Route Table

Destination	Target	
10.0.0.0/16	local	← VPC-internal stays local
0.0.0.0/0	nat-xxxxxxx	← Internet traffic → NAT Gateway

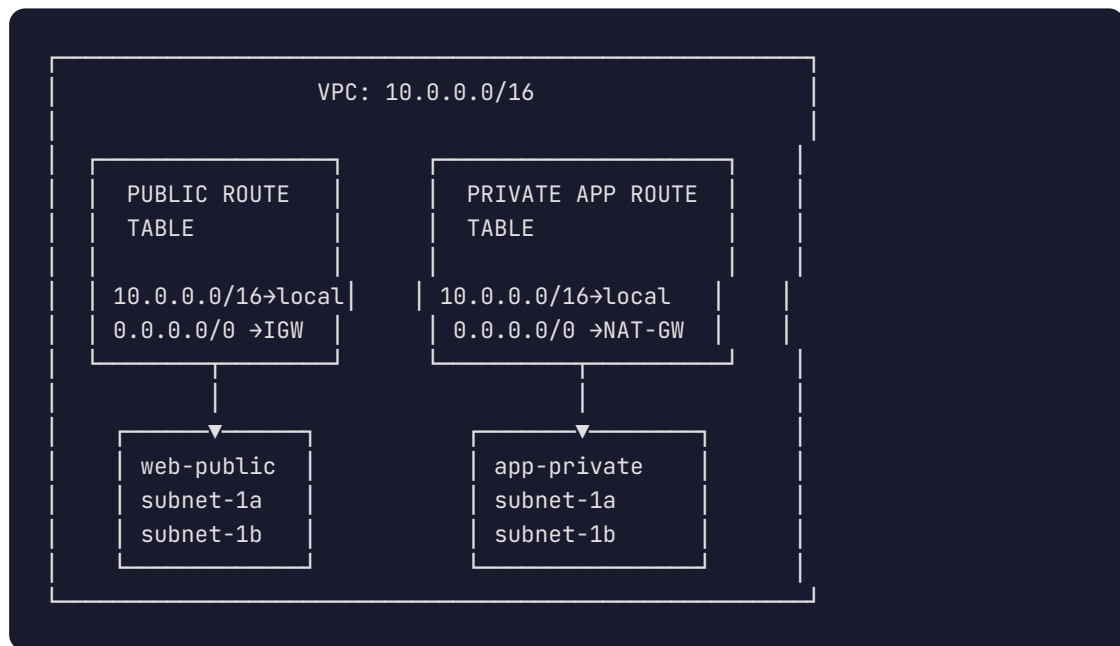
Associations: app-private-subnet-1a, app-private-subnet-1b

3. Database Route Table

Destination	Target	
10.0.0.0/16	local	← VPC-internal stays local
0.0.0.0/0	nat-xxxxxxx	← Internet traffic → NAT Gateway

Associations: data-private-subnet-1a, data-private-subnet-1b

ASCII Diagram — Route Tables



The "local" Route

Every route table always has a `local` route for your VPC CIDR. This cannot be deleted and enables all subnets within the VPC to communicate with each other by default.

8. Security Groups

Definition

A **Security Group** acts as a virtual firewall for your EC2 instances. It controls inbound and outbound traffic at the **instance level**.

Key Characteristics

- **Stateful** — if you allow inbound traffic, the response is automatically allowed outbound (no need to add explicit outbound rules for responses)
- **Allow only** — you can only add ALLOW rules, not DENY rules
- **Source can be another Security Group** — this is powerful for layered security
- Applied at the **network interface** level (per instance)
- Changes take effect **immediately**

Security Group Chaining (The Power Feature)

Instead of allowing traffic from an IP range, you can allow traffic **from another security group**. This means only instances with that specific security group can connect — regardless of IP address.

```
Public ALB (Public-ALB-SG)
  ↓ HTTP port 80 allowed from Public-ALB-SG
Web Instances (Web-SG)
  ↓ HTTP port 80 allowed from Web-SG
Internal ALB (Internal-ALB-SG)
  ↓ TCP port 4000 allowed from Internal-ALB-SG
App Instances (App-SG)
  ↓ MySQL port 3306 allowed from App-SG
Database (DB-SG)
```

This creates a **zero-trust, least-privilege** network where each layer only accepts traffic from the layer directly above it.

9. Task — Production Three-Tier Architecture on AWS

Architecture Overview

This task builds a **production-grade, highly available, three-tier web application** on AWS using everything you learned above.

```
Architecture:
├─ Web Tier    → NGINX (React frontend, reverse proxy)
├─ App Tier    → Node.js (REST API on port 4000)
└─ DB Tier     → MySQL (isolated database)
```

Production VPC Details

Component	Value
VPC CIDR	10.0.0.0/16
Region	ap-south-1 (Mumbai)
Availability Zones	ap-south-1a, ap-south-1b
Public ALB DNS	ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com
Internal ALB DNS	internal-Internal-ALB-1006516812.ap-south-1.elb.amazonaws.com

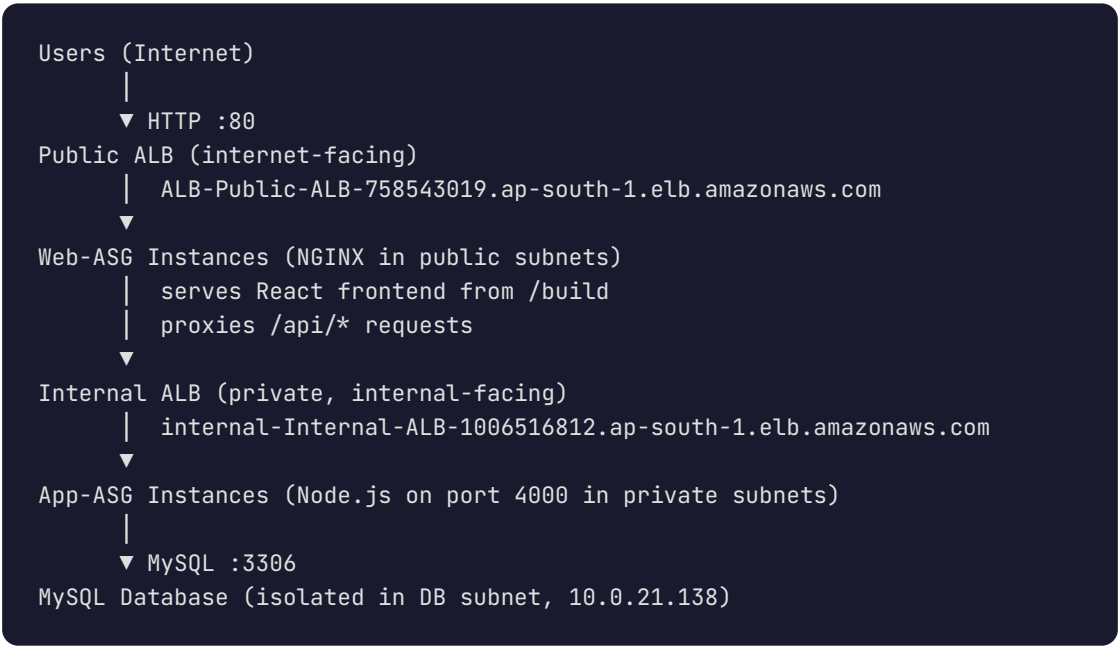
Subnet Layout

Subnet Name	Type	AZ	CIDR
web-public-subnet-1a	Public	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	Public	ap-south-1b	10.0.2.0/24
app-private-subnet-1a	Private (App)	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	Private (App)	ap-south-1b	10.0.4.0/24
data-private-subnet-1a	Private (DB)	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	Private (DB)	ap-south-1b	10.0.6.0/24

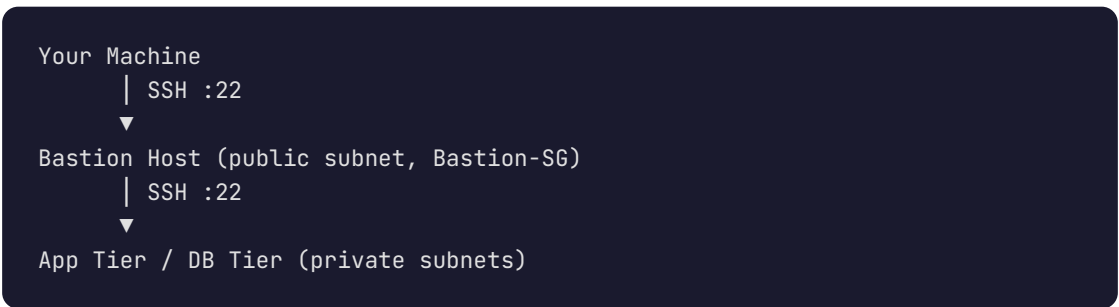
Full Architecture Diagram

The diagram above shows the complete three-tier architecture with all networking components, security groups, load balancers, auto scaling groups, and traffic flow.

Traffic Flow — How Requests Move



SSH Access Flow



Step-by-Step Implementation

Step 1 — Create VPC

AWS Console → VPC → Create VPC

Setting	Value
Name	Production VPC
CIDR Block	10.0.0.0/16
Tenancy	Default

Step 2 — Create Internet Gateway

VPC → Internet Gateways → Create Internet Gateway

Setting	Value
Name Tag	Production-IGW

After creation: **Actions → Attach to VPC → Production VPC**

Step 3 — Create Public Subnets

VPC → Subnets → Create Subnet → Select Production VPC

Subnet Name	Availability Zone	CIDR Block
web-public-subnet-1a	ap-south-1a	10.0.1.0/24
web-public-subnet-1b	ap-south-1b	10.0.2.0/24

After creation: Select each subnet → **Actions → Edit Subnet Settings → Enable Auto-assign Public IPv4**

Step 4 — Create App Private Subnets

Subnet Name	Availability Zone	CIDR Block
app-private-subnet-1a	ap-south-1a	10.0.3.0/24
app-private-subnet-1b	ap-south-1b	10.0.4.0/24

Do NOT enable auto-assign public IP for private subnets.

Step 5 — Create Database Private Subnets

Subnet Name	Availability Zone	CIDR Block
data-private-subnet-1a	ap-south-1a	10.0.5.0/24
data-private-subnet-1b	ap-south-1b	10.0.6.0/24

Step 6 — Create NAT Gateway

VPC → NAT Gateways → Create NAT Gateway

Setting	Value
Name	Production-NAT
Subnet	web-public-subnet-1a (must be public!)
Elastic IP	Click "Allocate Elastic IP"

Wait for NAT Gateway status to become **Available** before proceeding (~1-2 minutes).

Purpose: Allows private subnet instances to access the internet for: - `apt update` — OS updates - `npm install` — Node.js packages - `git clone` — pulling code - Any outbound internet calls

Step 7 — Create Route Tables

VPC → Route Tables → Create Route Table

1. Public Route Table

Setting	Value
Name	Public-RT
VPC	Production VPC

Routes:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-IGW

Subnet Associations: web-public-subnet-1a, web-public-subnet-1b

2. Private Route Table (App Tier)

Setting	Value
Name	Private-App-RT
VPC	Production VPC

Routes:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: app-private-subnet-1a, app-private-subnet-1b

3. Database Route Table

Setting	Value
Name	Database-RT
VPC	Production VPC

Routes:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Production-NAT

Subnet Associations: data-private-subnet-1a, data-private-subnet-1b

Step 8 — Create Security Groups

EC2 → Security Groups → Create Security Group

1. Bastion-SG — Used by Bastion Host

Type	Protocol	Port	Source
SSH	TCP	22	0.0.0.0/0

2. Public-ALB-SG — Used by Public ALB

Type	Protocol	Port	Source
HTTP	TCP	80	0.0.0.0/0

3. Web-SG — Used by Web Tier EC2 Instances

Type	Protocol	Port	Source
HTTP	TCP	80	Public-ALB-SG
SSH	TCP	22	Bastion-SG

4. Internal-ALB-SG — Used by Internal ALB

Type	Protocol	Port	Source
HTTP	TCP	80	Web-SG

5. App-SG — Used by App Tier Instances

Type	Protocol	Port	Source
Custom TCP	TCP	4000	Internal-ALB-SG
SSH	TCP	22	Bastion-SG

6. DB-SG — Used by Database Instance

Type	Protocol	Port	Source
MySQL/Aurora	TCP	3306	App-SG
SSH	TCP	22	Bastion-SG

Security Group Reference Map

Bastion Host

Public ALB

Web Instances

Internal ALB

App Instances

Database

—uses→

Bastion-SG

Public-ALB-SG

Web-SG

Internal-ALB-SG

App-SG

DB-SG

Step 9 — Create Bastion Host

EC2 → Launch Instance

Setting	Value
Name	Bastion Host
AMI	Ubuntu Server 22.04 LTS
Subnet	web-public-subnet-1a
Security Group	Bastion-SG
Auto-assign Public IP	Enabled

Purpose: SSH gateway to reach App Tier and Database Tier instances that are in private subnets.

Step 10 — Create Internal ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Setting	Value
Name	Internal-ALB
Scheme	Internal
Type	Application Load Balancer
Subnets	app-private-subnet-1a, app-private-subnet-1b
Security Group	Internal-ALB-SG

Step 11 — Create App Target Group

EC2 → Target Groups → Create Target Group

Setting	Value
Name	App-TG
Protocol	HTTP
Port	4000
Target Type	Instance
Health Check Protocol	HTTP
Health Check Path	/health

Step 12 — Launch Database EC2 Instance

EC2 → Launch Instance

Setting	Value
AMI	Ubuntu Server 22.04 LTS
Subnet	data-private-subnet-1a
Security Group	DB-SG
Private IP	Assigned by AWS (noted as 10.0.21.138)

Step 13 — Install and Configure MySQL

SSH into the DB instance via Bastion Host, then run:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install MySQL
sudo apt install mysql-server -y

# Secure MySQL and create database
sudo mysql
```

Inside MySQL prompt:

```
CREATE DATABASE webappdb;

CREATE USER 'appuser'@'%' IDENTIFIED BY 'YourStrongPassword123!';

GRANT ALL PRIVILEGES ON webappdb.* TO 'appuser'@'%';

FLUSH PRIVILEGES;

EXIT;
```

Allow remote connections:

```
# Edit MySQL config
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf

# Change this line:
bind-address = 127.0.0.1

# To this:
bind-address = 0.0.0.0
```

Restart and enable MySQL:

```
sudo systemctl restart mysql
sudo systemctl enable mysql
```

Step 14 — Launch App Tier Testing Instance

EC2 → Launch Instance

Setting	Value
AMI	Ubuntu Server 22.04 LTS
Subnet	app-private-subnet-1a
Security Group	App-SG

Step 15 — Install Node.js and PM2

SSH into App instance via Bastion, then:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Node.js 16
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -
sudo apt install -y nodejs git

# Install PM2 (process manager for Node.js)
sudo npm install -g pm2
```

PM2 keeps your Node.js app running in the background and auto-restarts it if it crashes.

Step 16 — Clone Project Repository

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

Step 17 — Configure Database Connection

```
nano aws-three-tier-web-architecture-workshop/application-code/app-tier/DbConfig.js
```

```
module.exports = Object.freeze({
  DB_HOST      : '10.0.21.138',
  DB_USER      : 'appuser',
  DB_PWD       : 'YourStrongPassword123!',
  DB_DATABASE  : 'webappdb'
});
```

Step 18 — Start App Tier

```
cd aws-three-tier-web-architecture-workshop/application-code/app-tier

# Install dependencies
npm install

# Start app with PM2
pm2 start index.js --name app-tier

# Save PM2 process list (survives reboots)
pm2 save
```

Verify the app is running:

```
curl http://localhost:4000/health
```

Expected response:

```
"This is the health check"
```

Step 19 — Register App Instance to App-TG

1. Go to **EC2** → **Target Groups** → **App-TG**
2. Click **Register Targets**
3. Select the App Testing Instance
4. Click **Include as pending below** → **Register pending targets**

5. Wait for health check status to show **Healthy**

Step 20 — Create Public ALB

[EC2](#) → [Load Balancers](#) → [Create Load Balancer](#) → [Application Load Balancer](#)

Setting	Value
Name	Public-ALB
Scheme	Internet-facing
Subnets	web-public-subnet-1a, web-public-subnet-1b
Security Group	Public-ALB-SG
Listener	HTTP :80

Step 21 — Create Web Target Group

[EC2](#) → [Target Groups](#) → [Create Target Group](#)

Setting	Value
Name	Web-TG
Protocol	HTTP
Port	80
Target Type	Instance
Health Check Protocol	HTTP
Health Check Path	/

Step 22 — Launch Web Tier Testing Instance

[EC2](#) → [Launch Instance](#)

Setting	Value
AMI	Ubuntu Server 22.04 LTS
Subnet	web-public-subnet-1a
Security Group	Web-SG
Auto-assign Public IP	Enabled

Step 23 — Install NGINX and Node.js

SSH directly into the web tier instance (it has a public IP):

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Node.js 16
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash -

# Install Node.js, NGINX, and Git
sudo apt install nodejs nginx git -y
```

Clone the repository:

```
git clone https://github.com/Asadkhanrtx/aws-three-tier-web-architecture-workshop.git
```

Step 24 — Build React Application

```
cd aws-three-tier-web-architecture-workshop/application-code/web-tier

# Install dependencies
npm install

# Build the production bundle
npm run build
```

This creates a `/build` folder containing the compiled React app ready to be served by NGINX.

Step 25 — Configure NGINX

```
sudo nano /etc/nginx/sites-available/default
```

Replace the entire file content with:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # Health check endpoint
    location /health {
        default_type text/html;
        return 200 "<!DOCTYPE html><p>Web Tier Health Check</p>\n";
    }

    # Serve React frontend
    location / {
        root /home/ubuntu/aws-three-tier-web-architecture-workshop/
application-code/web-tier/build;
        index index.html index.htm;
        try_files $uri /index.html;
    }

    # Reverse proxy: forward /api/ requests to Internal ALB → App Tier
    location /api/ {
        proxy_pass http://internal-Internal-ALB-1006516812.ap-
south-1.elb.amazonaws.com;
    }
}
```

How NGINX Connects Web and App Tiers

```
Browser request: GET /api/health
|
▼
NGINX on Web Instance
| location /api/ matches
▼
proxy_pass → Internal ALB DNS
| internal-Internal-ALB-1006516812.ap-south-1.elb.amazonaws.com
▼
Internal ALB routes to App-TG
|
▼
Node.js App Instance :4000
|
▼
Response flows back through the same path
```

Step 26 — Restart NGINX

```
# Test configuration for syntax errors
sudo nginx -t

# Restart NGINX
sudo systemctl restart nginx

# Enable NGINX to start on boot
sudo systemctl enable nginx
```

Step 27 — Register Web Tier to Web-TG

1. Go to **EC2** → **Target Groups** → **Web-TG**
2. Click **Register Targets**
3. Select the Web Tier Testing Instance
4. Register and wait for status: **Healthy**

Step 28 — Final Testing

Frontend Access:

```
http://ALB-Public-ALB-758543019.ap-south-1.elb.amazonaws.com
```

Backend Health Check (from Web instance):

```
curl http://localhost/api/health
```

Expected:

```
"This is the health check"
```

Database Connectivity: Use the DB Demo Page in the React app to verify end-to-end database read/write.

Step 29 — Create AMIs

Create Amazon Machine Images (AMIs) from your tested instances so Auto Scaling can launch identical copies.

EC2 → Instances → Select Instance → Actions → Image and Templates → Create Image

Instance	AMI Name	Option
Web Tier Instance	ubuntu-web-tier-ami-v1	Enable "Reboot instance"
App Tier Instance	ubuntu-app-tier-ami-v1	Enable "Reboot instance"

Enabling reboot ensures a consistent filesystem state in the AMI.

Step 30 — Create Launch Templates

EC2 → Launch Templates → Create Launch Template

1. Web Launch Template (Web-LT)

Setting	Value
Name	Web-LT
AMI	ubuntu-web-tier-ami-v1
Security Group	Web-SG

2. App Launch Template (App-LT)

Setting	Value
Name	App-LT
AMI	ubuntu-app-tier-ami-v1
Security Group	App-SG

Step 31 — Create Auto Scaling Groups

EC2 → Auto Scaling Groups → Create Auto Scaling Group

Web-ASG

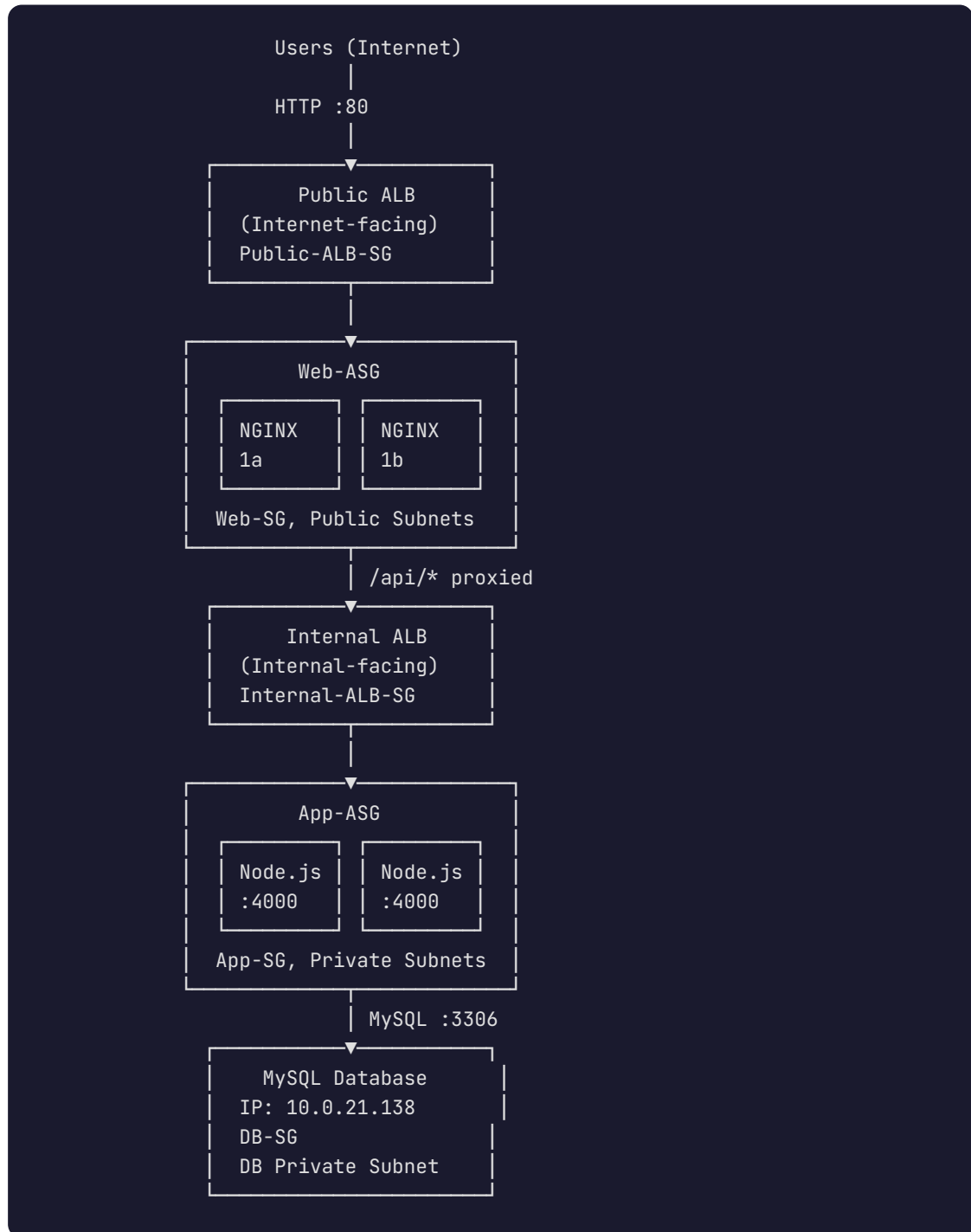
Setting	Value
Name	Web-ASG
Launch Template	Web-LT
Subnets	web-public-subnet-1a, web-public-subnet-1b
Target Group	Web-TG
Desired Capacity	2
Minimum Capacity	2
Maximum Capacity	4

App-ASG

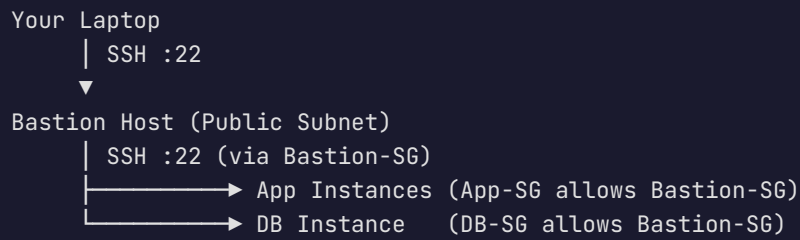
Setting	Value
Name	App-ASG
Launch Template	App-LT
Subnets	app-private-subnet-1a, app-private-subnet-1b
Target Group	App-TG
Desired Capacity	2
Minimum Capacity	2
Maximum Capacity	4

Final Architecture Summary

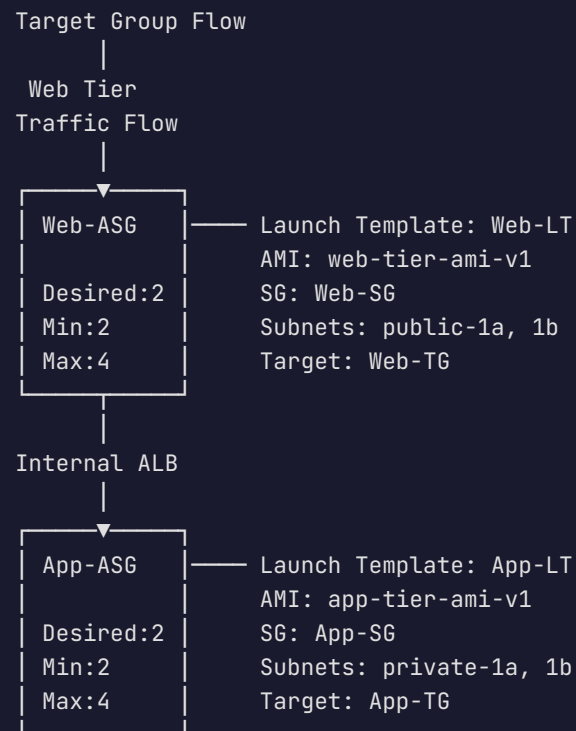
Complete Traffic Flow Diagram



SSH Access Diagram



Auto Scaling Group Summary



Production Benefits of This Architecture

Benefit	How Achieved
High Availability	Multi-AZ deployment (1a + 1b for every tier)
Fault Tolerance	Auto Scaling replaces failed instances automatically
Security	Private backend — app and DB never exposed to internet
Scalability	ASG scales from 2 to 4 instances based on load
Secure SSH Access	Single controlled entry point via Bastion Host
Reverse Proxy	NGINX separates frontend concerns from backend routing
Database Isolation	DB in its own subnet, only App-SG can reach port 3306
Load Distribution	Two ALBs distribute load at both web and app tiers
Enterprise Networking	Proper CIDR planning with room to grow (10.0.0.0/16)
Zero Internet Exposure for DB	No route to IGW from DB subnet — outbound via NAT only

Documentation prepared for AWS Networking Case Study — VPC Module